

JWT Security Best Practices



Scott Brady

Identity Consultant & Engineering Manager

@scottbrady91

www.scottbrady91.com



Overview



How attackers take advantage of JOSE's cryptographic agility

Always use standardized claim types

How to revoke some or all tokens

Life beyond bearer tokens with proof of possession

Alternatives to JWTs



Trust No One, Rely on Existing Relationships



JWT headers are hints, not instructions.



Cryptographic Agility



**Can support
many algorithms
at once**



**Can support
new algorithms
without changes
to the format**



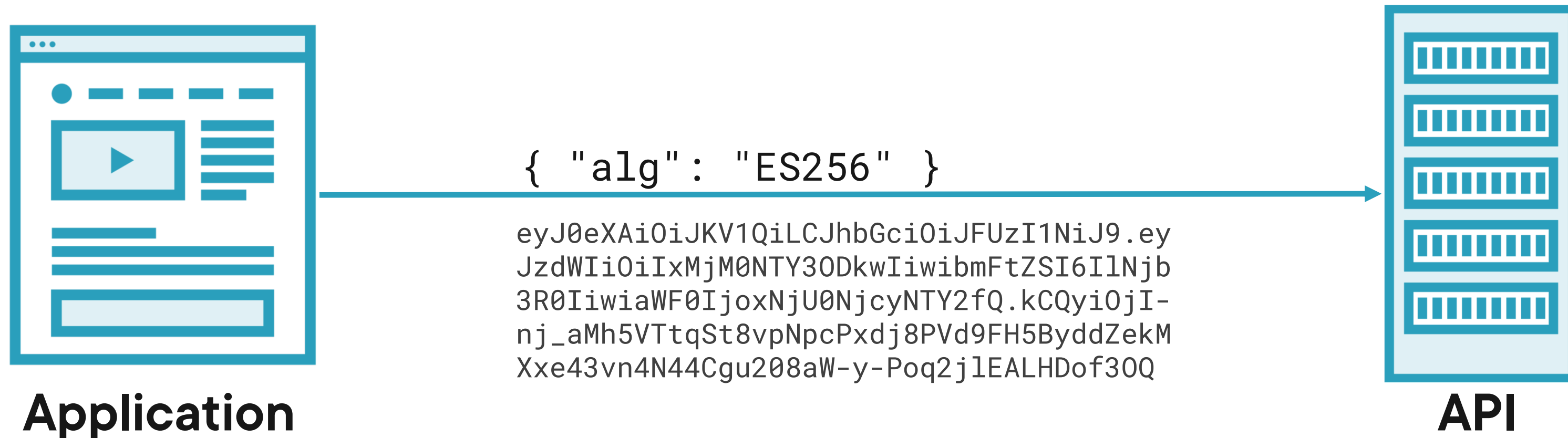
**Achieved using
the algorithm
header**



**Similar to TLS
handshakes**



Valid Signed Token



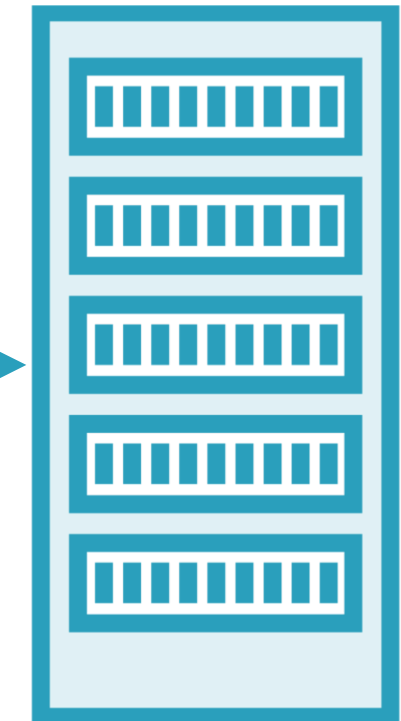
Valid Unsigned Token



Attacker

`{ "alg": "none" }`

`eyJ0eXAiOiJKV1QiLCJhbGciOiJIub251In0.eyJ
zdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6I
R0IiwiaWF0IjoxNjU0NjcyNTY2fQ.`



API



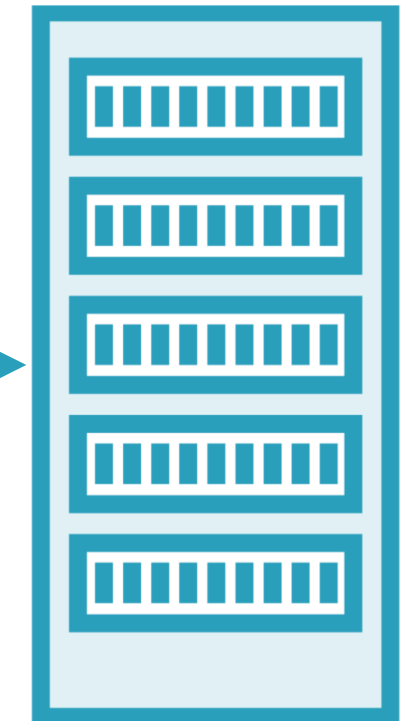
Valid Unsigned Token



Attacker

`{ "alg": "noNe" }`

`eyJ0eXAiOiJKV1QiLCJhbGciOiJub05lIn0.eyJ
zdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6I
R0IiwiaWF0IjoxNjU0NjcyNTY2fQ.`



API



Days Since an Unsigned JWT Vulnerability

**It has been 185 days since the
last alg=none JWT
vulnerability.**

Conecte SUS, the official app of the Brazilian Ministry of Health, does not validate JWT signatures when validating vaccine certificates offline. It doesn't accept alg=none, but it might as well.

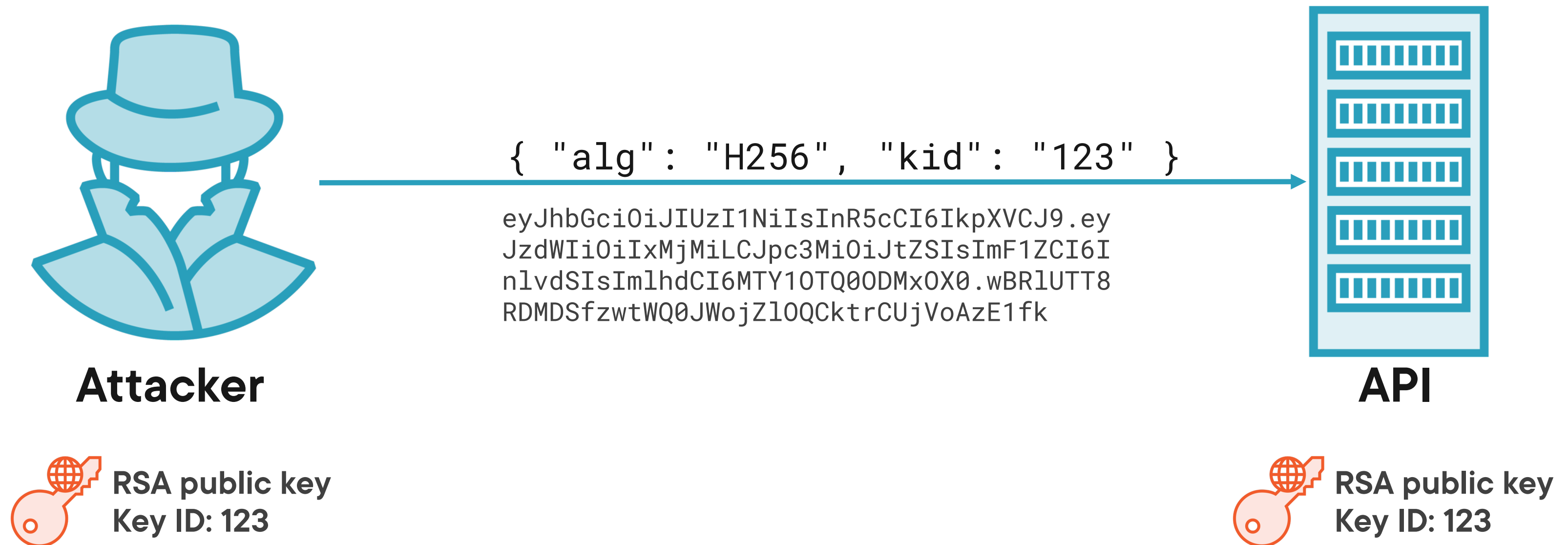
made by [zofrex](#)

out of date? [@me on Twitter](#)

<https://www.howmanydayssinceajwtalgnonevuln.com/>



Algorithm Downgrade



“It’ll never happen to me!”

Famous last words



Validating Algorithms and Keys



Do I support this algorithm?



Do I know this key?



Does this key make sense for this algorithm?



Does this key belong to the token's issuer?



```
{
  "alg": "ES256",
  "jwk": {
    "kty": "EC",
    "x": "18tFrhx-34tV3hRICRDY9zCkD1pBhF42UQUfWVAWBFs",
    "y": "9VE4jf_0k_o64zbTT1cuNJajHmt6v9TDVrU0CdvGRDA",
    "crv": "P-256"
  }
}
```

Embedded Keys

The key that the JWT wants you to use during validation.

Could be used as a hint (same as key ID header)

```
{  
  "alg": "ES256",  
  "kid": "f7811b04be0844c0b8c77250504623d9",  
  "jku": "https://idp.example.com/jwks.json"  
}
```

Key Lookup

A URL containing the keys to validate this token.

Must be validated against an allow list.

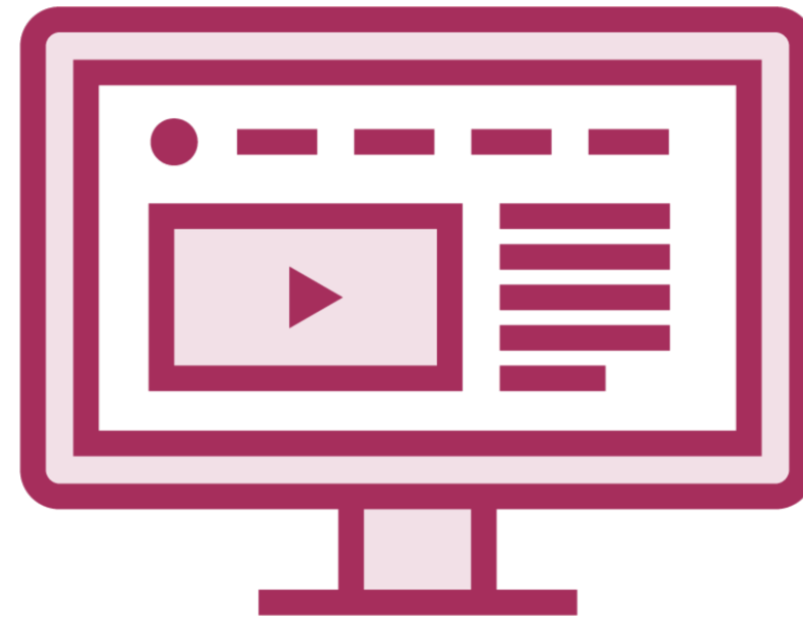
Rely on Standardized Claims to Prevent Misuse



Standard Claims



Issuer



Audience



Subject

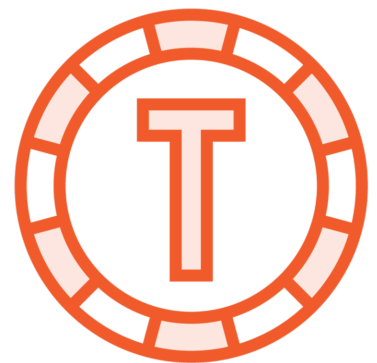


Lifetime

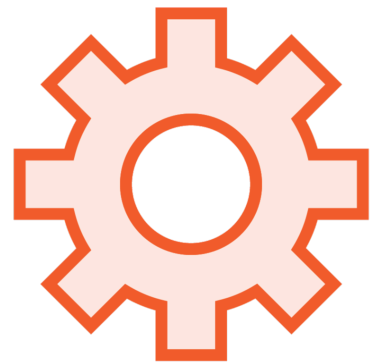
Payload Validation



Take advantage of JWT library payload validation



Enforce your own rules with token types



Always use a standard claim when possible



JWT Revocation



Only the token validator can
enforce revocation.



Token Revocation

How to block a single JWT

How to invalidate all JWTs



Token Blocklists

JWT ID

df7ff3bad71840bb9afd3e3a9d2eaa5b
b9864b5306944df7a69abc59caec6514
ae1532026ab843829777ca3003070fbe
a521e259c00f4abd9e3b3c6f17d49433
2599da1f015445c5879fd0d8653037c7

.

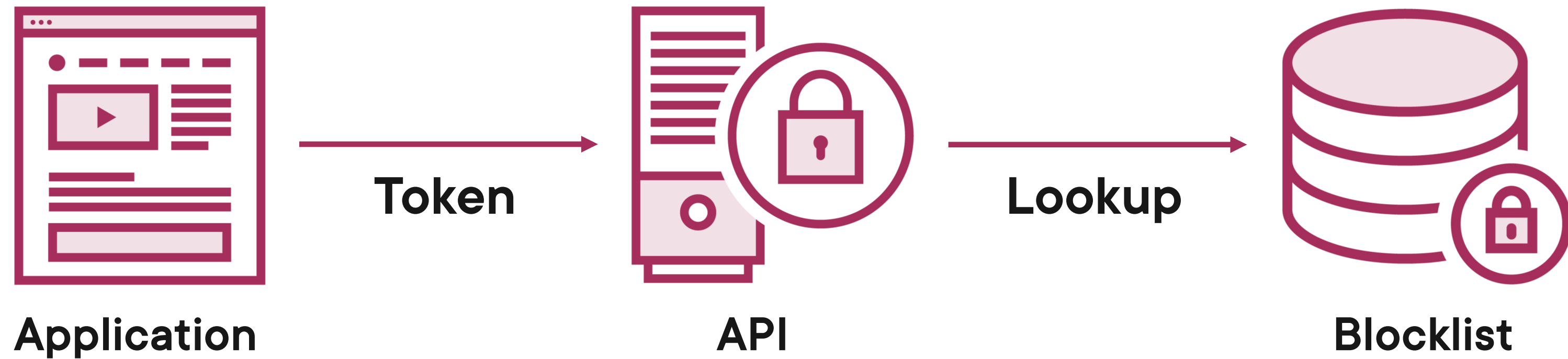
.

.

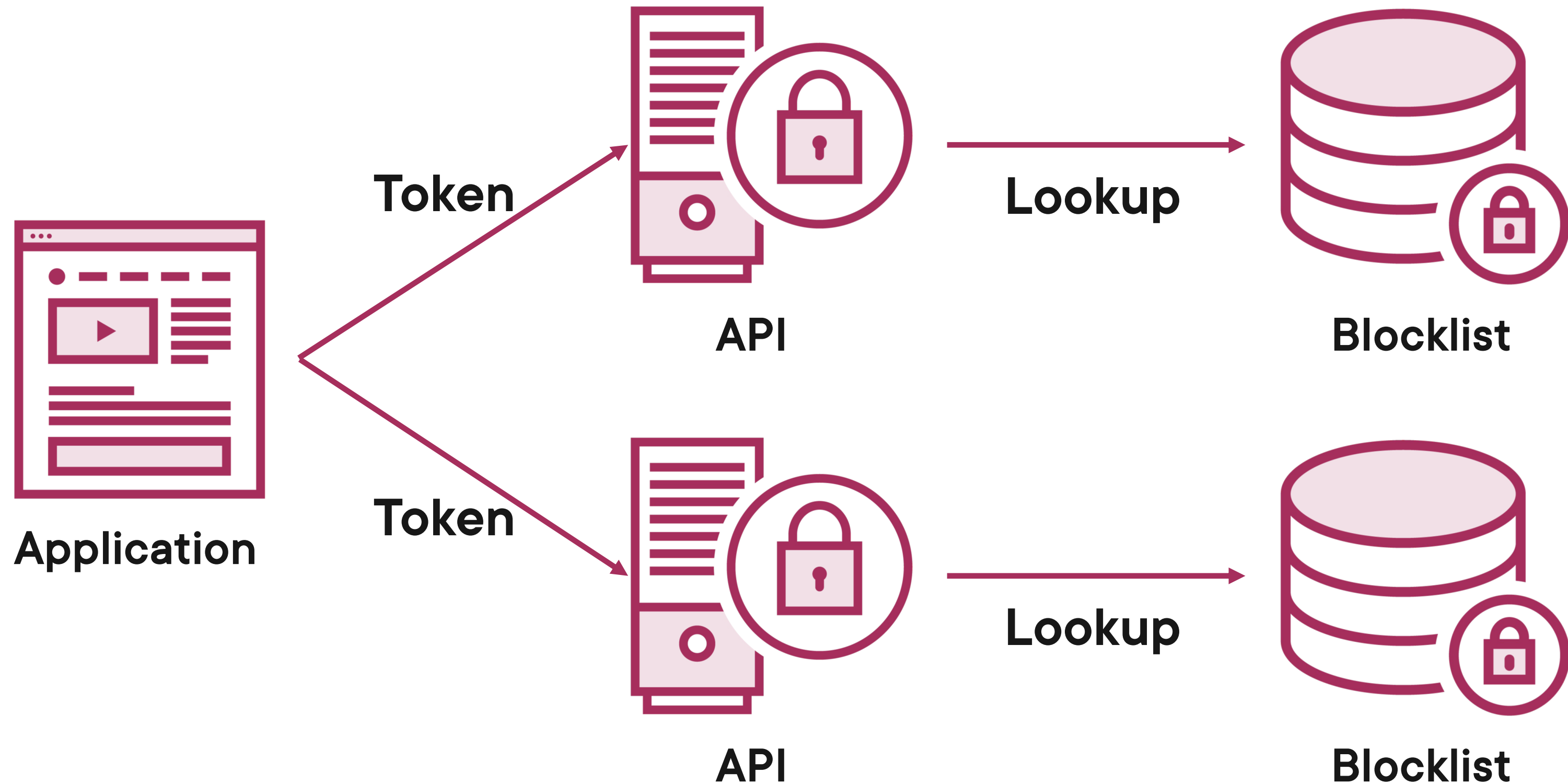
8d43eeffa2e864988ab9b79da33e5b530



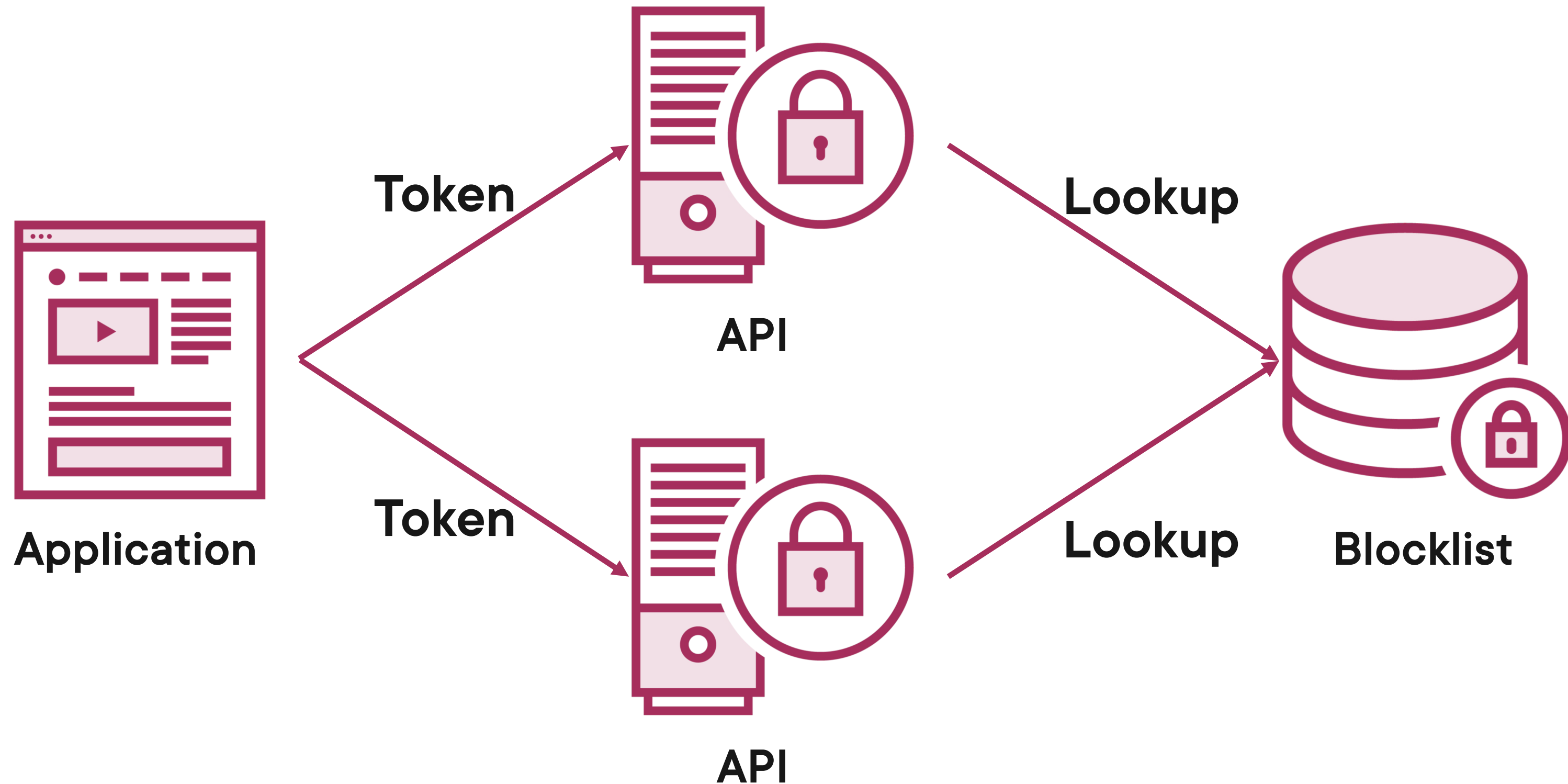
Token Blocklists



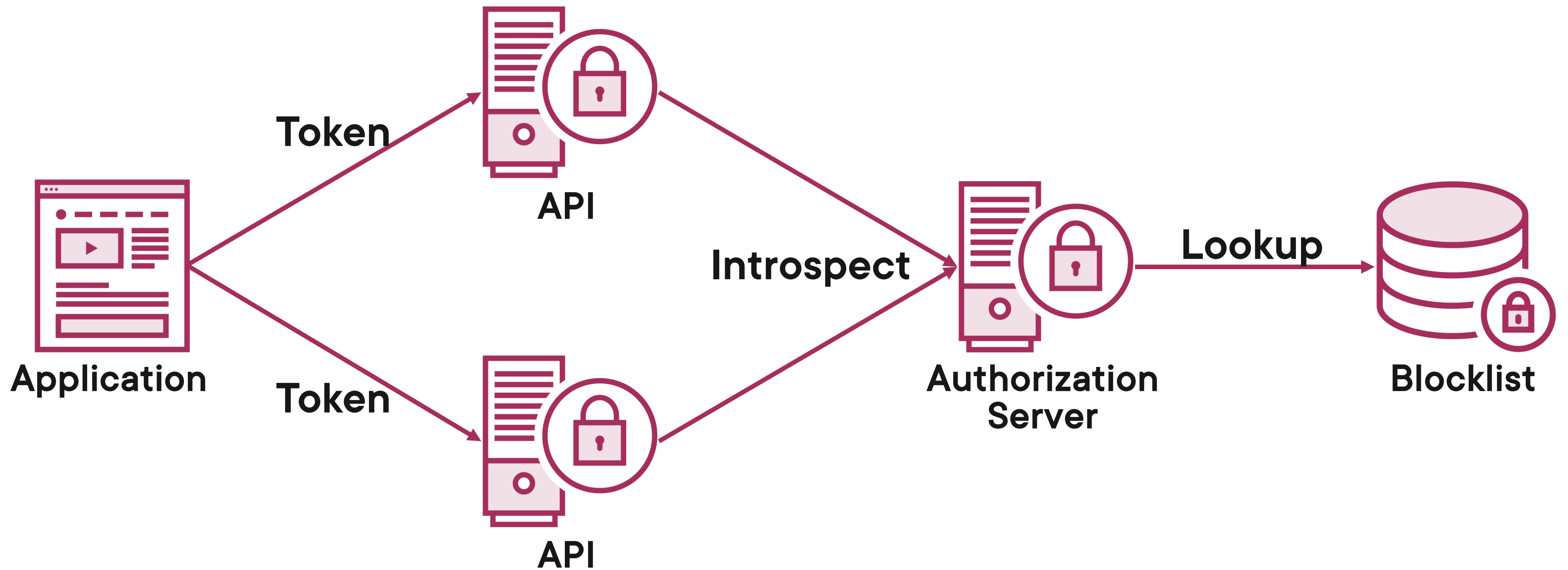
Token Blocklists



Token Blocklists



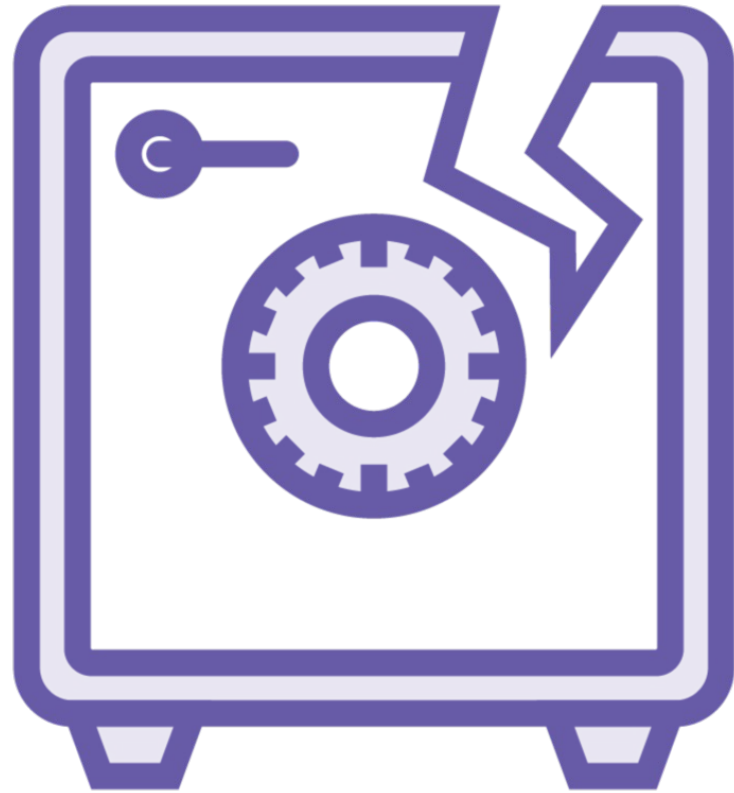
Token Blocklists



Are you still gaining the
benefits of stateless JWTs?



When to Invalidate All JWTs

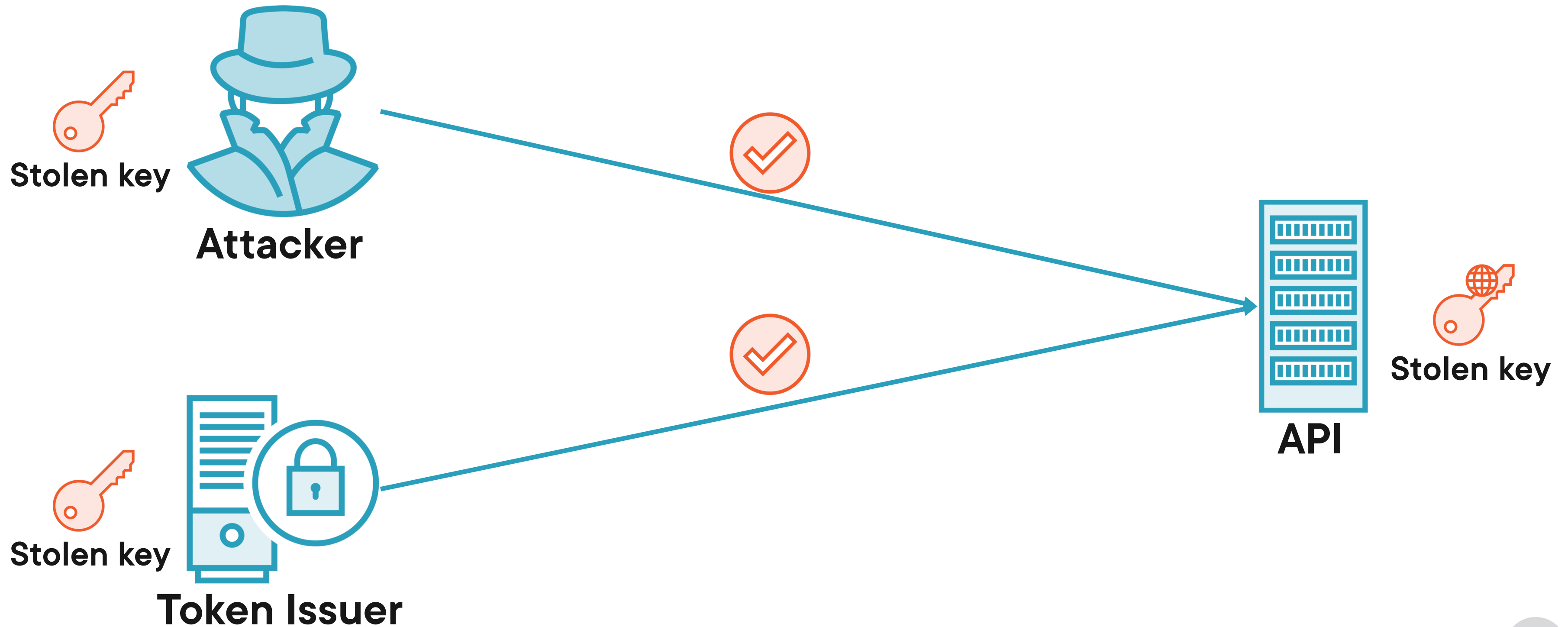


Tokens have been stolen

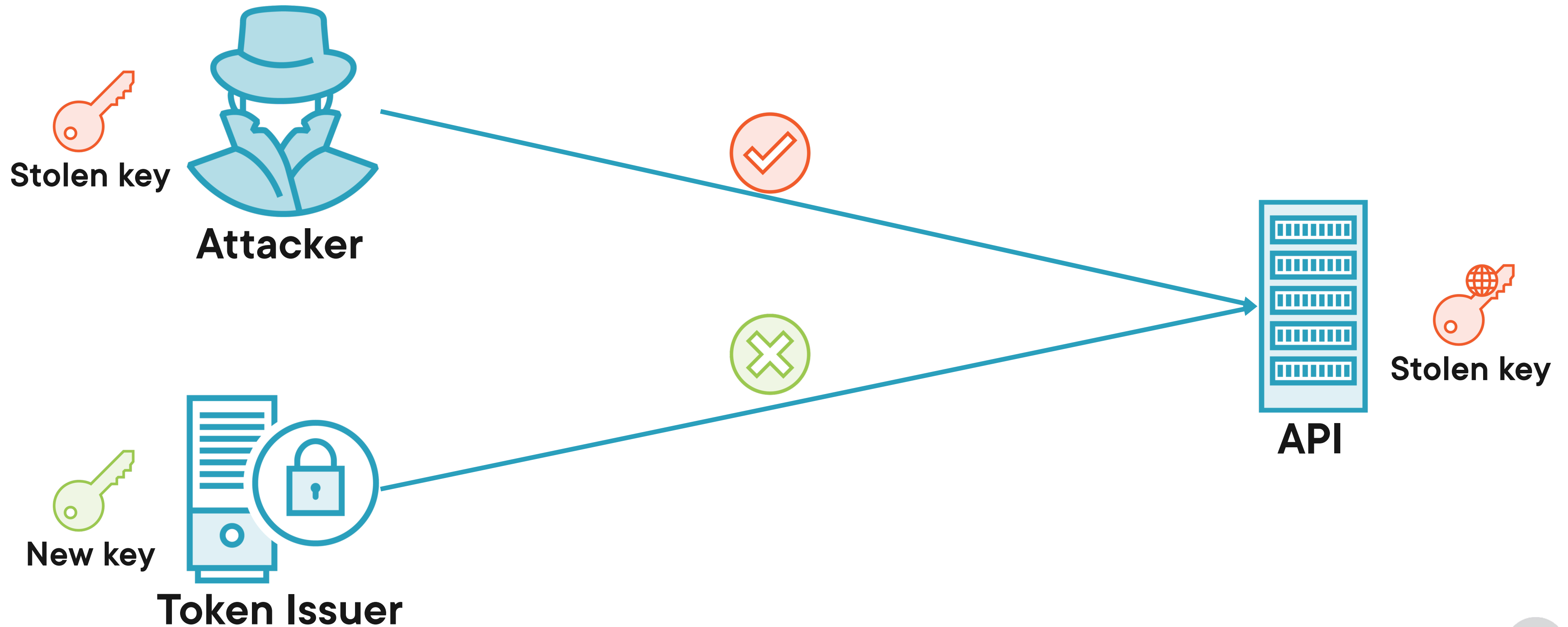


**Tokens are being created by a
malicious party**

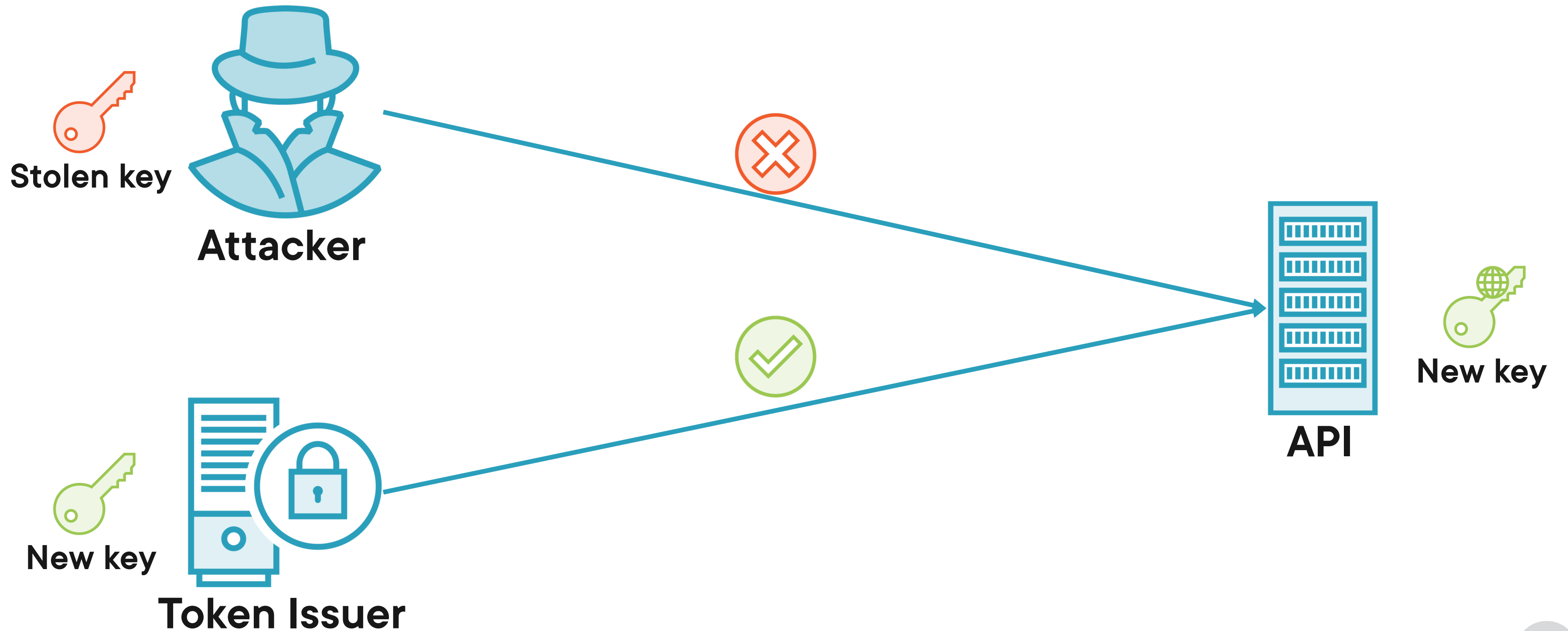
Key Rollover



Key Rollover



Key Rollover



Proof of Possession



Bearer Tokens



Proof of Possession



Proof of Possession



Very hard to implement



Hard to work with untrusted applications, such as mobile and browser-based apps (e.g. SPAs)



OAuth has some promising solutions

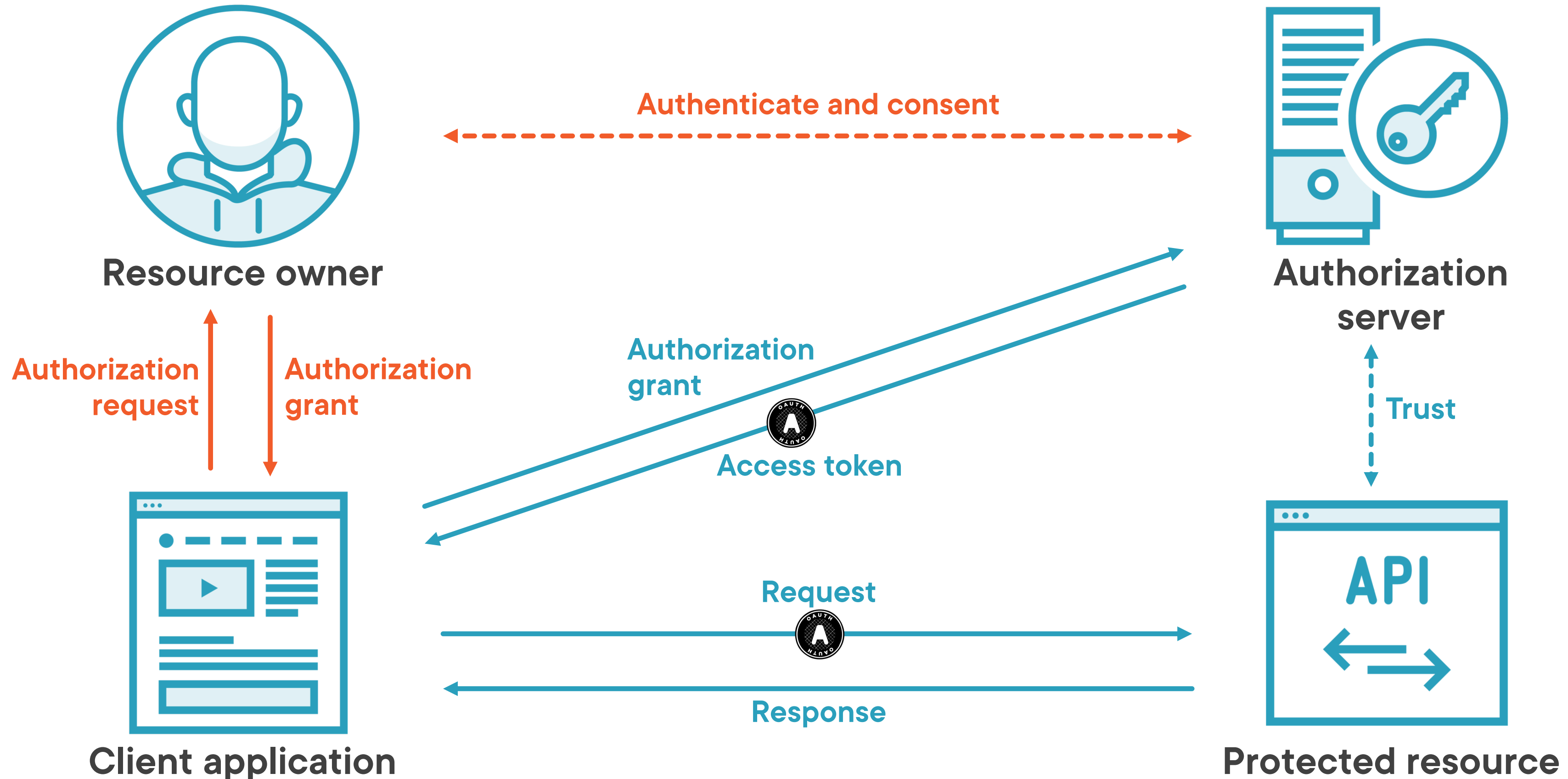


RFC 8705

OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens



Mutual TLS (mTLS) for Client Authentication



Mutual TLS (mTLS) for Client Authentication



**Authorization
server**

Authorization
grant
+ mTLS



Client application

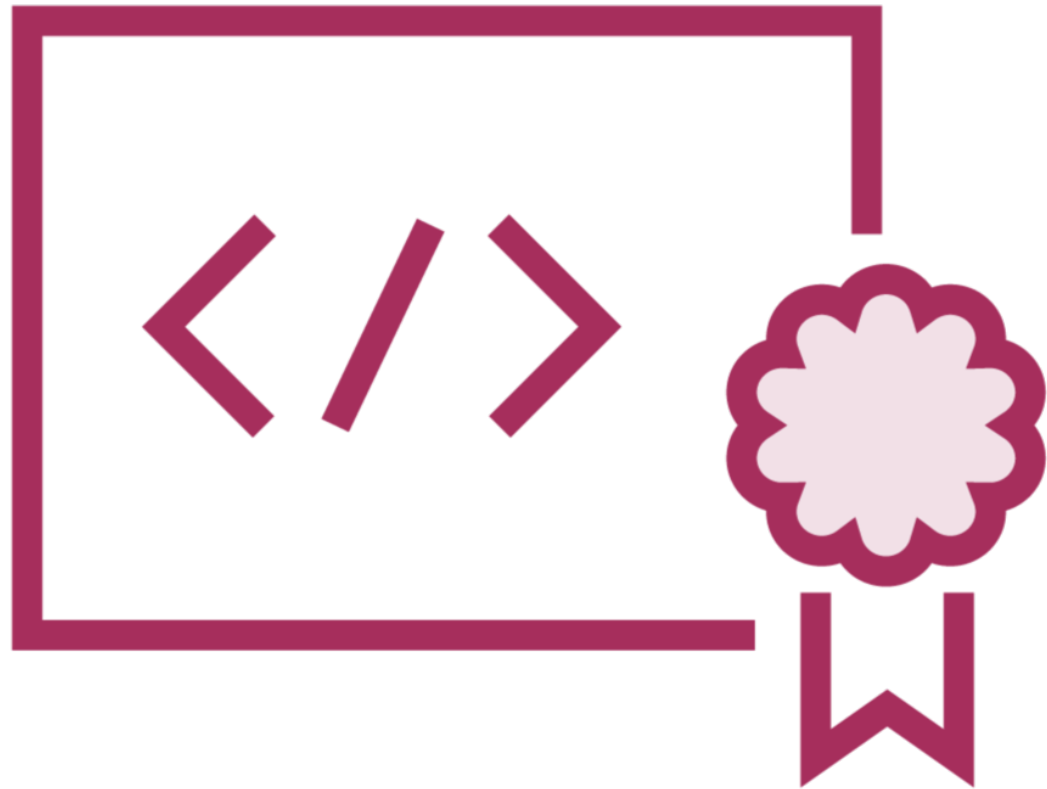


```
{
  "iss": "https://idp.example.com",
  "aud": "https://api.example.com",
  "sub": "b3bfa8d95a17468292a87995e36552dd",
  "exp": 1656060914,
  "iat": 1656060314,
  "cnf": {
    "x5t#S256": "bwcK0esc3ACC3DB2Y5_1ESsXE8o91tc05089jdN-dg2"
  }
}
```

JWT Certificate Thumbprint Confirmation Method

Describes the certificate that must be used for mTLS alongside this token.

mTLS Limitations



Requires support for client certificates



Not suitable for some application types

draft-ietf-oauth-dpop

**OAuth 2.0 Demonstrating Proof-of-Possession at the Application Layer
(DPoP)**



DPoP Access Token Request



Authorization
grant
+ DPoP Proof



Client application



```
{
  "typ": "dpop+jwt",
  "alg": "ES256",
  "jwk": { "kty": "EC", "x": "l8tFrhx-34tV3hRICRDY9zCkD1pBhF42UQUfWVAWBFs",
    "y": "9VE4jf_0k_o64zbTTlcuNJajHmt6v9TDVrU0CdvGRDA", "crv": "P-256" }
}.
{
  "jti": "-BwC3ESc6acc2lTc",
  "htm": "POST",
  "htu": "https://idp.example.com/token",
  "iat": 1656060314
}
```

DPoP Proof JWT for Token Request

Tells the authorization server what key to bind an access token to.

```
{
  "iss": "https://idp.example.com",
  "aud": "https://api.example.com",
  "sub": "b3bfa8d95a17468292a87995e36552dd",
  "exp": 1656060914,
  "iat": 1656060314,
  "cnf": {
    "jkt": "0Zc0C0RZNYy-DWpqq30jZyJGHTN0d2Hg1BV3uiguA4I"
  }
}
```

JWK Thumbprint Confirmation Method

Describes the public key for the DPoP proof that must be sent alongside this token.

DPoP Resource Server Request



```
{
  "typ": "dpop+jwt",
  "alg": "ES256",
  "jwk": { "kty": "EC", "x": "l8tFrhx-34tV3hRICRDY9zCkD1pBhF42UQUfWVAWBFs",
    "y": "9VE4jf_0k_o64zbTTlcuNJajHmt6v9TDVrU0CdvGRDA", "crv": "P-256" }
}.
{
  "jti": "-ATd4ESc7acS4rwB",
  "htm": "POST",
  "htu": "https://api.example.com/protectedresource",
  "iat": 1656060414
}
```

DPoP Proof JWT for Protected Resource

Proves knowledge of the private key to the resource server.

If it can create a JWT, it can
use DPoP.



Alternatives to JWTs



JWT Best Practices



Use restricted profiles



**Use with standardized
protocols**



**Follow best practices
(RFC 8725)**



PASETO

Platform-Agnostic Security Token



Cryptographic Agility



~~Cryptographic Agility~~ Versioning



v4.public.eyJpc3MiOiJodHRwczovL2lkC5leGFtcGxlLmNvbSIsICJhdWQiOiAiaHR0cHM6Ly9hcGkuZXhhbXBsZS5jb20iLCAic3ViIjoieYjNiZmE4ZDk1YTE3NDY4MjkyYTg3OTk1ZTM2NTUyZGQiLCAiZXhwIjogIjIwMjMtMDYtMTU0MTk2Mzk2MTQuMDU0Mzk0WiIsICJpYXQiOiAiMjAyMi0wNi0xOVQxOTozMzoxNC4wNTQzOTRaIiwgImp0aSI6ICI4ODI0NzE1ZTJiZDY0OTU0YmU3ZGIzZWZjVjMwIiwiaWF0IjoiMjAyMi0wNi0xOVQxOTozMzoxNC4wNTQzOTRaIiwgImNSESIF5g8idp52ajbFZNQ2FNe_jDCGEAwH79xt9skelN_25yaFTAM

PASETO Format

Version, purpose, payload, and an optional footer

1. *Journal of the American Medical Association*, 1997; 278: 1019-1024.

```
{  
  "iss": "https://idp.example.com",  
  "aud": "https://api.example.com",  
  "sub": "b3bfa8d95a17468292a87995e36552dd",  
  "exp": "2022-06-19T19:39:14.054394Z",  
  "iat": "2022-06-19T19:33:14.054394Z",  
  "jti": "8824715e2bd64954be7db3ec0f5c1b4a"  
}
```

Payload

Mostly the same as JOSE but sometimes uses different value types.

PASETO Purpose

local

public



Versioning

Sign

Given a message `m`, Ed25519 secret key `sk`, and optional footer `f` (which defaults to empty string), and an optional implicit assertion `i` (which defaults to empty string):

1. Before signing, first assert that the key being used is intended for use with `v4.public` tokens, and is the secret key of the intended keypair. See [Algorithm Lucidity](#) for more information.
2. Set `h` to `v4.public`.
3. Pack `h`, `m`, `f`, and `i` together using [PAE](#) (pre-authentication encoding). We'll call this `m2`.
4. Sign `m2` using Ed25519 `sk`. We'll call this `sig`.

```
sig = crypto_sign_detached(  
    message = m2,  
    private_key = sk  
);
```

5. If `f` is:
 - Empty: return "`h` || base64url(`m` || `sig`)"
 - Non-empty: return "`h` || base64url(`m` || `sig`) || `.` || base64url(`f`)"
 - ...where || means "concatenate"
 - Note: `base64url()` means Base64url from RFC 4648 without `=` padding.

<https://github.com/paseto-standard/paseto-spec/>



```
{  
  "kid": "5eeb1d5cf5f8462d98332c4a68583367"  
}
```

Footer

You still need to validate the key ID against the algorithm.

PASETO



**Does not improve standard claim
validation**



**Does not improve inadequate
library design**

Versioning == Cryptographic Agility?



Version (1B) | Timestamp (4B) | Nonce (24B) | Ciphertext (*B) | Tag (16B)

Branca Format

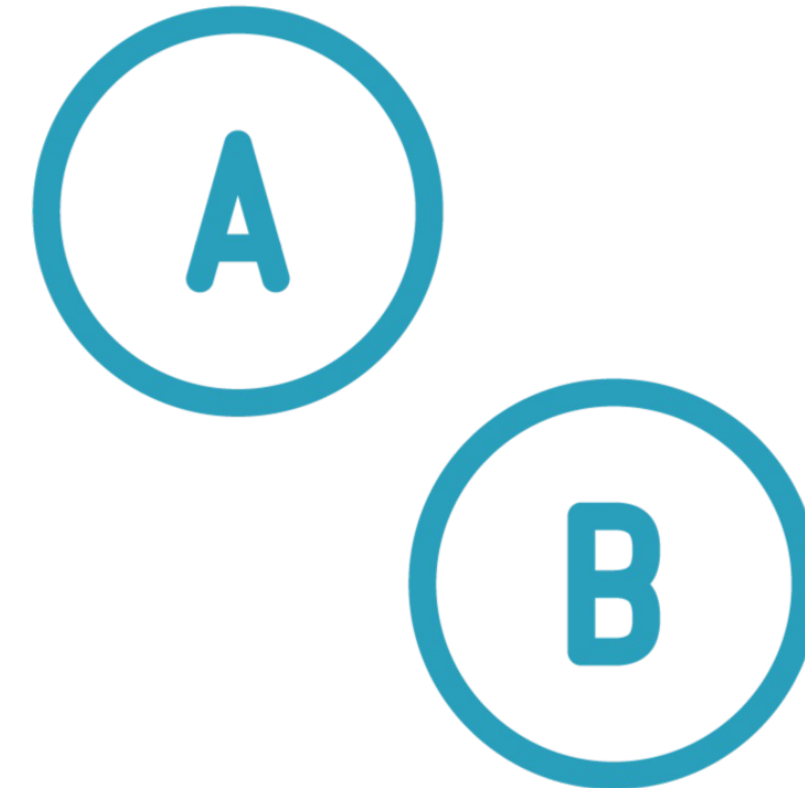
Base62 encoded string – no delimiters.

Consider Branca tokens over local HMAC tokens.

JWT Alternatives



**Often lack lessons learned from
JWT libraries**



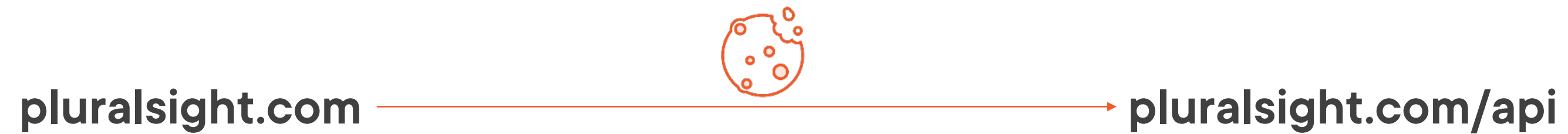
**Very low adoption compared to
JWTs**

Consider not using tokens at
all.



SameSite Cookies

Set-Cookie: id=38afes7a8; Secure; HttpOnly



SameSite Cookies

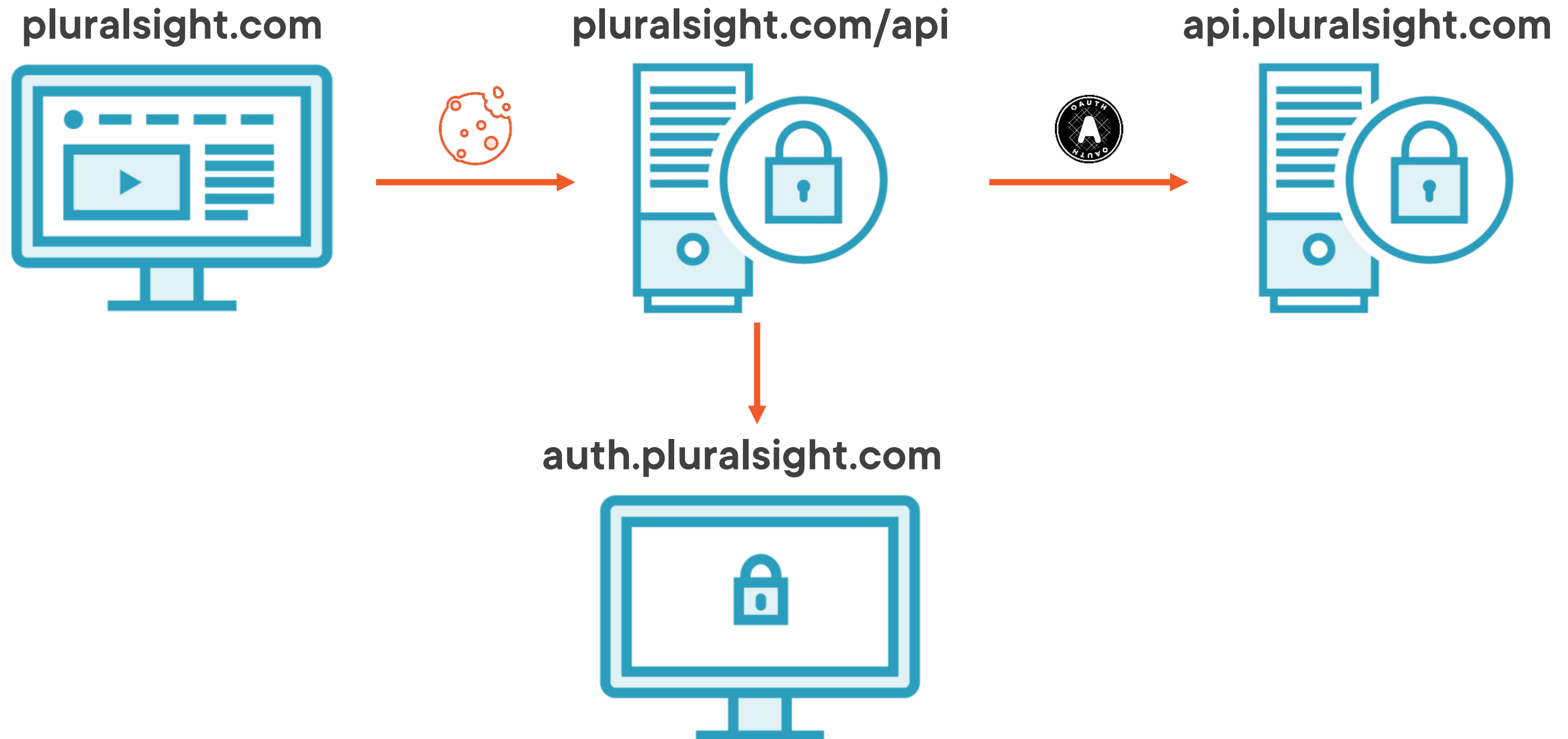
Set-Cookie: id=38afes7a8; Secure; HttpOnly; **SameSite=Strict**

pluralsight.com  pluralsight.com/api

scottbrady91.com  401: Unauthorized pluralsight.com/api



Backend for Frontend



Learn More



“Getting Started with OAuth” – Pluralsight



“Alternatives to JWTs” – scottbrady91.com



“Key-driven cryptographic agility” – neilmadden.blog



Summary



How attackers take advantage of JOSE's cryptographic agility

Always use standardized claim types

How to revoke some or all tokens

Life beyond bearer tokens with proof of possession

Alternatives to JWTs



RFC 8725

JSON Web Token Best Current Practices

